# LLM Gateway — Database, Persistence & Usage Logging

Interview preparation · V1 · PostgreSQL + SQLAlchemy + Alembic, failure-safe usage logs

## 1. Database Foundation

PostgreSQL, SQLAlchemy, and the schema.

### Q1. What does your database store and what are the tables?

EASY

**Answer:** Three tables: users (the client behind a key), api_keys (gateway keys stored as SHA-256 hashes, with is_active and a user_id foreign key), and requests (one usage record per gateway call). Only metadata is stored — never prompts or responses.

**Simple explanation:** Who the clients are, their hashed keys, and a log of each request's metadata.

| Table | Key columns | Purpose |
|---|---|---|
| users | id, name, created_at | The client/account |
| api_keys | id, user_id, key_hash, is_active, created_at | Hashed gateway keys |
| requests | request_id, user_id, api_key_id, model, provider, tokens, latency_ms, status, error_code | Usage log |

*Interview tip: Emphasize 'metadata only, no prompt/response content' — it shows privacy awareness.*

### Q2. Why PostgreSQL, and why SQLAlchemy as the ORM?

EASY

**Answer:** PostgreSQL is a reliable, widely-used relational database that fits structured, related data (users -> keys -> requests) with foreign keys. SQLAlchemy 2.0 gives typed ORM models, a clean session API, and pairs with Alembic for migrations.

**Simple explanation:** A solid relational DB for related records, with a mature Python ORM on top.

**Project example:** Relationships: a User has many ApiKeys; requests reference both user_id and api_key_id so usage can be attributed.

***Likely follow-ups:***

- Why not NoSQL? (the data is relational and small; joins and constraints are a natural fit)

### Q3. Why synchronous SQLAlchemy in an async FastAPI app?

HARD

**Answer:** The DB work is tiny and infrequent (one key lookup per request, one insert for logging). FastAPI runs sync dependencies in a threadpool, so they don't block the event loop. Sync also keeps Alembic simple (standard env.py, no async/greenlet complexity). Async drivers would add complexity for negligible benefit at this scale.

**Simple explanation:** Small, rare queries + threadpool = sync is simpler and fast enough.

**Project example:** The provider calls (the real bottleneck) stay async via httpx; only the light DB layer is sync.

*Interview tip: This is a trade-off question — lead with 'the DB isn't the bottleneck'.*

## 2. Migrations with Alembic

Schema as versioned, repeatable migrations.

### Q4. How do you manage the database schema?

MEDIUM

**Answer:** With Alembic migrations. The schema is defined by SQLAlchemy models; Alembic autogenerates migration files that are applied with 'alembic upgrade head'. The app never calls create_all() at startup — migrations are the single source of truth.

**Simple explanation:** Versioned migration files, applied in order, describe the exact schema.

**Project example:** There are two migrations: the initial users/api_keys/requests schema, and one that makes requests.provider / provider_model nullable (for error rows).

***Likely follow-ups:***

- Why not create_all()? (no version history, no safe upgrades, no team reproducibility)

### Q5. In Docker, when do migrations run, and what if they fail?

MEDIUM

**Answer:** The backend container's entrypoint runs 'alembic upgrade head' BEFORE starting uvicorn, and uses set -e so a failed migration aborts the container. That guarantees the app never serves against a broken or outdated schema.

**Simple explanation:** Migrate first; if migration fails, don't start serving.

```
set -e
alembic upgrade head
exec uvicorn app.main:app --host 0.0.0.0 --port 8000
```

*Interview tip: The 'set -e, migrate-then-serve' detail is a strong reliability talking point.*

## 3. Usage Logging

Recording metadata without breaking requests.

### Q6. How and where is usage logging done?

MEDIUM

**Answer:** At the route boundary, not inside GatewayService. The route times the call with a monotonic clock, then a small usage service writes a requests row via a repository. On success it logs model, provider, provider_model, tokens, latency, status=success. On a post-auth error it logs status=error and the error_code, then re-raises.

**Simple explanation:** The route measures and records each call's metadata after it finishes.

**Project example:** It reuses the same request_id that goes in the response header, so a log row and a client's X-Request-ID line up exactly.

***Likely follow-ups:***

- Why at the route, not in GatewayService? (the route has identity, DB session, timing, and the final result)

### Q7. What happens if the logging/database write fails?

HARD

**Answer:** Logging is failure-safe: the write is wrapped so any DB error is caught, rolled back, and downgraded to a server-side warning. It never changes the response the client already earned — a successful chat stays 200, and a provider error stays its original 502. Logging is observability, not part of the request's success contract.

**Simple explanation:** If logging breaks, the user still gets their correct answer or error.

**Project example:** A test monkeypatches the log write to throw and asserts the endpoint still returns 200 (success) / 502 (provider error) unchanged.

*Interview tip: Great 'what if X fails' answer: logging must never mask the real outcome.*

---

## Q8. What do you deliberately NOT store, and why?

EASY

**Answer:** No prompts, no model responses, no conversation history, and no raw keys. The requests table has no column for message content. This keeps sensitive user content out of the database by default; it is usage/metadata logging, not conversation storage.

**Simple explanation:** We log 'what happened', not 'what was said'.

**Project example:** Verified the requests table has no prompt/response/content/key columns; only metadata plus an api_key_id foreign key.

*Interview tip: Privacy-by-default is a strong signal; mention tokens are stored but not the text.*

---